- Registers `ExchangeRatesClient` as a transient service in DI.
- Controls the creation of `ExchangeRatesClient` so that whenever a new instance is required, a pooled `HttpClient` is configured as defined in the `AddHttpClient<T>` lambda method.

TIP You can think of a typed client as a wrapper around a named client. I'm a big fan of this approach, as it encapsulates all the logic for interacting with a remote service in one place. It also avoids the magic strings that you use with named clients, removing the possibility of typos.

Another option when registering typed clients is to register an interface in addition to the implementation. This is often good practice, as it makes it much easier to test consuming code. If the typed client in listing 33.5 implemented the interface `IExchangeRatesClient`, you could register the interface and typed client implementation using

```
builder.Services.AddHttpClient<IExchangeRatesClient, ExchangeRatesClient>()
```

You could then inject this into consuming code using the interface type

```
app.MapGet("/", async (IExchangeRatesClient ratesClient) =>
    await ratesClient.GetLatestRates());
```

Another common pattern is to not provide any configuration for the typed client in the `AddHttpClient()` call. Instead, you could place that logic in the constructor of your `ExchangeRatesClient` using the injected `HttpClient`: